부록 · 리눅스 명령어

Linux sed · awk 유용한 실전 예제 30개

JSP · Spring Boot · properties · conf · 로그 · Linux 서버 관리 실습

1. sed와 awk의 핵심 차이

sed는 스트림 편집기로 문자열 치환, 특정 줄 삭제, 삽입, 범위 출력 등에 강함. awk는 조건에 맞는 행 선택, 필드 분리, 줄 번호 표시, 계산 및 통계 처리에 강함.

기억법:

```bash
sed = 편집(치환·삭제·삽입), awk = 분석(조건·범위·필드·계산).
```

구분	주요 역할	대표 문법	실무 용도
sed	텍스트 편집	s/OLD/NEW/g	설정 변경, 줄 삭제, 삽입
awk	텍스트 분석	조건 {동작}	로그 분석, 필드 추출, 통계

2. sed 실전 예제 15개

1. 문자열 치환

파일을 수정하지 않고 8080을 8081로 바꾼 결과를 화면에 표시함.

```bash
sed 's/8080/8081/g' application.properties
```

2. 실제 파일 수정

-i 옵션으로 파일 자체를 수정함.

```bash
sed -i 's/8080/8081/g' application.properties
```

3. 백업 후 수정

원본을 .bak 파일로 보관하고 수정함.

```bash
sed -i.bak 's/8080/8081/g' application.properties
```

4. 특정 줄 삭제

10번째 줄을 삭제함.

```bash
sed -i.bak '10d' member.jsp
```

5. 범위 삭제

10~20번째 줄을 선택 삭제함.

```bash
sed -i.bak '10,20d' member.jsp
```

6. ERROR 줄 제거

ERROR가 포함된 줄을 제외하여 출력함.

```bash
sed '/ERROR/d' app.log
```

7. 빈 줄 삭제

공백만 있거나 완전히 빈 줄을 제거함.

```bash
sed '/^[[:space:]]*$/d' file.txt
```

8. 주석 줄 제거

#으로 시작하는 설정 주석을 제외함.

```bash
sed '/^[[:space:]]*#/d' application.properties
```

9. 특정 줄만 출력

10번째 줄만 출력함.

```bash
sed -n '10p' member.jsp
```

10. 특정 범위 출력

10~30번째 줄만 편집기 없이 확인함.

```bash
sed -n '10,30p' member.jsp
```

11. 특정 설정 출력

datasource 관련 줄만 출력함.

```bash
sed -n '/spring.datasource/p' application.properties
```

12. 설정 뒤에 내용 삽입

server.port 다음 줄에 server.address를 추가한 결과를 보여줌.

```bash
sed '/server.port/a server.address=0.0.0.0' application.properties
```

13. 설정 앞에 주석 삽입

server.port 앞에 설명 주석을 삽입함.

```bash
sed '/server.port/i # Server Setting' application.properties
```

14. 여러 공백을 하나로 정리

연속된 공백을 하나의 공백으로 정리함.

```bash
sed 's/[[:space:]][[:space:]]*/ /g' file.txt
```

15. IP 주소 일괄 변경

MariaDB나 서버 IP 주소 변경 등에 활용함.

```bash
sed -i.bak 's/192\.168\.0\.10/SERVER_IP/g' application.properties
```

3. sed 추가 실무 응용

기존 server.port 값이 무엇이든 8081로 변경:

```bash
sed -i.bak 's/^server\.port=.*/server.port=8081/' application.properties
```

JSP의 회원가입 문구를 회원등록으로 변경:

```bash
sed -i.bak 's/회원가입/회원등록/g' member.jsp
```

주석과 빈 줄을 동시에 제외:

```bash
sed -e '/^[[:space:]]*#/d' -e '/^[[:space:]]*$/d' application.properties
```

HTML/JSP 태그를 제거하고 텍스트만 추출:

```bash
echo '<h1>Hello Linux</h1>' | sed 's/<[^>]*>//g'
```

여러 properties 파일의 localhost를 일괄 변경:

```bash
find . -name "*.properties" -exec sed -i.bak 's/localhost/SERVER_IP/g' {} +
```

4. awk 실전 예제 15개

16. 전체 내용 출력

파일의 모든 줄을 출력함.

```bash
awk '{print}' file.txt
```

17. 줄 번호와 내용 출력

NR을 이용해 줄 번호를 붙임.

```bash
awk '{print NR ": " $0}' file.txt
```

18. ERROR 검색

ERROR가 있는 줄만 선택함.

```bash
awk '/ERROR/{print}' app.log
```

19. ERROR 줄 번호 표시

원본 로그의 ERROR 줄 번호와 내용을 표시함.

```bash
awk '/ERROR/{print NR ": " $0}' app.log
```

20. ERROR/WARN 동시 검색

ERROR 또는 WARN이 있는 줄을 찾음.

```bash
awk '/ERROR|WARN/{print NR ": " $0}' app.log
```

21. 특정 줄 범위 출력

10~30줄만 선택하여 출력함.

```bash
awk 'NR>=10 && NR<=30 {print NR ": " $0}' member.jsp
```

22. 첫 번째 필드 출력

공백 기준 첫 번째 필드만 출력함.

```bash
awk '{print $1}' file.txt
```

23. 여러 필드 출력

첫 번째와 세 번째 필드를 선택함.

```bash
awk '{print $1,$3}' file.txt
```

24. 구분자 지정

:을 구분자로 사용하여 Linux 사용자명만 추출함.

```bash
awk -F: '{print $1}' /etc/passwd
```

25. ERROR 개수 계산

ERROR가 발생한 총 건수를 계산함.

```bash
awk '/ERROR/{n++} END{print n+0}' app.log
```

26. ERROR/WARN 통계

두 종류의 로그 건수를 한 번에 계산함.

```bash
awk '/ERROR/{e++} /WARN/{w++} END{print "ERROR=",e+0,"WARN=",w+0}' app.log
```

27. 숫자 합계

첫 번째 필드의 숫자를 모두 합산함.

```bash
awk '{sum+=$1} END{print "합계=",sum}' data.txt
```

28. 숫자 평균

첫 번째 필드의 평균을 계산함.

```bash
awk '{sum+=$1} END{if(NR) print "평균=",sum/NR}' data.txt
```

29. 특정 값 이상 선택

세 번째 필드 값이 80보다 큰 행만 선택함.

```bash
awk '$3>80 {print}' data.txt
```

30. JSP form 영역 추출

```bash
<form>부터 </form>까지의 코드 영역만 추출함.
awk '/<form/,/<\/form>/{print NR ": " $0}' member.jsp
```

5. awk 추가 실무 응용

ERROR와 WARN을 구분하여 출력:

```bash
awk '
/ERROR/ {print "[ERROR]", NR, $0}
/WARN/  {print "[WARN ]", NR, $0}
' app.log
```

Linux 사용자명과 UID 출력:

```bash
awk -F: '{
    print "USER=" $1, "UID=" $3
}' /etc/passwd
```

UID 1000 이상 계정 확인:

```bash
awk -F: '$3 >= 1000 {
    print $1, $3
}' /etc/passwd
```

디스크 사용률 80% 이상 경고:

```bash
df -P | awk '
NR > 1 {
    use=$5
    gsub("%","",use)
    if (use >= 80)
        print "WARNING:", $1, use "%"
}'
```

JSP script 영역 추출:

```bash
awk '/<script/,/<\/script>/ {
    print NR ": " $0
}' member.jsp
```

6. sed + awk + tee 병합 예제

application.properties에서 주석과 빈 줄을 제거한 뒤 실제 설정에 번호를 붙여 출력:

```bash
sed -e '/^[[:space:]]*#/d' -e '/^[[:space:]]*$/d' application.properties |
awk '{print NR ": " $0}'
```

ERROR/WARN을 추출하고 번호를 붙여 보고서 파일에도 저장:

```bash
sed -n '/ERROR\|WARN/p' app.log |
awk '{print NR ": " $0}' |
tee error-warn-report.txt
```

안전한 설정 변경의 권장 순서:

```bash
grep -n 'server.port' application.properties
cp application.properties application.properties.bak
sed -i 's/^server\.port=.*/server.port=8081/' application.properties
grep -n 'server.port' application.properties | tee change-result.txt
```

7. 실무 안전 원칙

- sed -i로 운영 파일을 수정하기 전에 sed 명령에서 -i를 빼고 변경 결과를 먼저 확인하는 습관이 좋음.
- 실제 수정 시 sed -i.bak 또는 cp 명령으로 원본을 백업하는 것이 안전함.
- find와 sed를 결합하면 여러 파일이 동시에 바뀌므로 grep/find로 대상 파일을 먼저 확인해야 함.
- awk는 원본 파일을 직접 수정하는 도구라기보다 검색, 필드 가공, 조건 분석, 집계에 활용하는 것이 이해하기 쉬움.
- 로그 및 설정 분석 결과를 남길 때 tee를 조합하면 화면 확인과 보고서 저장을 동시에 할 수 있음.
